

AI 工具使用详情

数学建模小组

2026 年 9 月 13 日

1 AI 工具说明和使用环节

本报告用于记录数学建模任务中人工智能工具的实际使用情况，便于说明模型建立、程序实现和论文图表制作过程。

本次任务中主要使用的生成式人工智能工具为 **Codex**，CLI 版本号为 0.153.4，所配置的模型为 **deepseek-V4.1-Flash**。Codex 在本机工作区中完成了以下辅助工作：

1. 阅读题目材料、附件说明、已有代码和结果文件；
2. 将自然语言问题整理为变量、假设、控制方程、边界条件和验证方案；
3. 编写和修改 Python 数值求解、敏感性分析、收敛检验和绘图程序；
4. 根据已有计算结果整理 Markdown 或 LaTeX 报告。

2 分工与协作原则

2.1 人机分工

1. **人工负责：**确定问题目标，判断模型是否合理，确认物理参数和边界条件，解释结果，决定是否采用某一方案。
2. **Codex 负责：**整理信息、提出备选路径、生成公式草稿、编写程序、绘制图表和执行重复性检查。
3. **共同完成：**根据数值结果修正模型，比较粗网格与细网格、不同时间步长和不同物性过渡方式下的结果。

2.2 使用边界

本报告不使用 Codex 代替实验数据或参考文献，也不把生成文本直接作为最终结论。对于需要引用来源、实验标定或竞赛规则判断的内容，仍以题目原文、附件数据和人工核验为准。

3 使用示例一：思路整理

3.1 任务背景

题目要求研究圆柱形药材在烘房环境中的温度和水分变化，并给出不同时刻、不同径向位置的温度与水分浓度。烘房温度变化分为两个阶段，不同阶段参数不同，需要精准区分两个阶段

3.2 提供给 Codex 的提示词

第二问里分为预热和恒温两个阶段，绘制烘房温度随时间变化的图像，并提出寻找阶段分界的方法，在图像里标注出升温期和平台期的分界。

3.3 Codex 的整理结果

在阶段识别方面，Codex 根据附件 1 的尾部平台数据建议使用中位数和 MAD 稳健统计确定恒温干燥起点，并通过滑动均值和斜率条件判定临界时刻。同时绘制了烘房温度随时间变化的图像，标注出升温期和平台期的分界。最后得到 $t_c = 6780$ s

3.4 论文对应位置

问题二 5.2.1.4 阶段分界时刻 t_c 的稳态统计判定

3.5 人工复核，修改与采纳

1. 观察烘房温度曲线，直观上确认阶段分界是否合理
2. 人工修改分界参数，进行敏感性分析
3. 人工检索 AI 提供方法的合理性，找到理论依据，最后采纳 AI 提供的方法

4 使用示例二：图片绘制

4.1 任务背景

第三问需要展示不同典型时刻药材含水量沿径向的变化，并突出含水量阈值

$$C_{\text{threshold}} = 0.15 \text{ kg/kg.} \quad (1)$$

图表必须正确读取 result3.xlsx，区分不同时刻，使用一致的单位和中文标签，并能同时输出可交互 HTML 与可用于论文的 PNG。

4.2 提供给 Codex 的提示词

请读取 `result3.xlsx`，绘制第三问典型时刻的含水量径向剖面。横轴为距药材轴线的距离，单位为 `cm`，范围 0 到 2；纵轴为含水量，单位为 `kg/kg`。选择 6、12、24、36、48、54 h 和烘干结束时刻，画出阈值 $C = 0.15$ 的虚线，并将达标区域做浅色标注。图题、坐标轴和悬浮提示使用中文，输出 PNG。

4.3 Codex 生成的绘图实现

Codex 生成了脚本，主要步骤为：使用 `openpyxl` 读取结果工作表；对每个径向列在目标时刻做线性插值；使用 `Plotly` 绘制曲线；添加阈值线和达标区域；再通过浏览器渲染流程导出 PNG。

下面是绘图核心逻辑的简化摘录：

Listing 1: Plotly 径向含水量剖面核心代码

```
1 figure.add_hline(  
2     y=THRESHOLD,  
3     line=dict(color="#C00000", width=2.4, dash="dash"),  
4     annotation_text="阈值 C=0.15 kg/kg",  
5 )  
6  
7 for hour in typical_times:  
8     profile = profile_at_time(hour, time_hour, concentration)  
9     figure.add_trace(  
10         go.Scatter(  
11             x=radii,  
12             y=profile,  
13             mode="lines+markers",  
14             name=f"{hour:.0f} h",  
15         )  
16     )  
17  
18 figure.update_xaxes(title="距药材轴线的距离 / cm", range=[0, 2])  
19 figure.update_yaxes(title="含水量 / (kg/kg)")  
20 figure.write_html("figures/plotly/q3_04_moisture_profiles.html")
```

4.4 生成结果

Codex 根据上述脚本生成了径向含水量剖面图。图中同时展示多个典型时刻，并使用红色虚线标出 $C = 0.15$ 的干燥终点阈值。

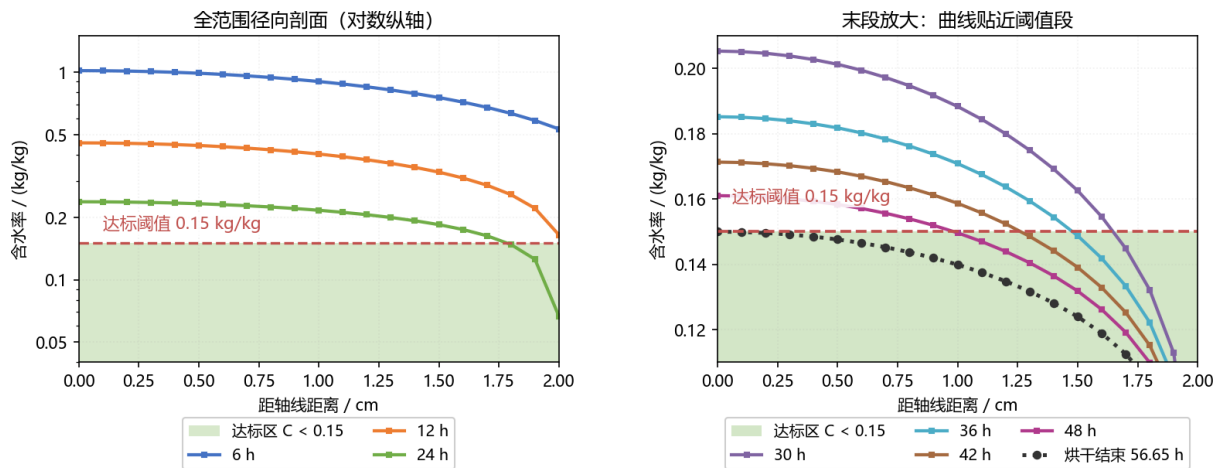


图 1: 第三间药材含水率时空分布与径向差异 (达标阈值 0.15 kg/kg)

4.5 论文对应位置

图 8: 第三间药材含水率时空分布与径向差异 (达标阈值 0.15 kg/kg)

4.6 人工复核，修改与采纳

1. 检查绘图脚本读取的工作表、时间和径向列是否与题目输出模板一致
2. 检查曲线是否与主模型结果文件和全局水分守恒检验一致
3. 人工修改图例、单位、有效数字和时间标签使得图片更加美观
4. 检验绘图脚本读取的数据和生成的图片，确认无误后采纳该图片

5 使用示例三：代码辅助

5.1 任务背景

长时干燥过程的数值计算需要同时处理变物性、二维轴对称几何、中心轴奇异性、所有外表面的 Robin 边界、两阶段物性过渡和长时间稳定推进。直接使用显式方法可能受到稳定时间步长限制，因此需要考虑守恒离散和隐式时间格式。

5.2 提供给 Codex 的提示词

请基于当前二维轴对称热湿模型编写 Python 求解程序。空间离散采用守恒有限体积格式，正确处理 $r = 0$ 的中心轴；圆柱侧面和两个端面使用第三类边界条件。时间推进采用自适应 BDF 算法求解隐式阶段方程。请输出 result3.xlsx，同时保存完整 NPZ 数据。

5.3 Codex 生成的程序结构

Codex 代码使用 `scipy.integrate.solve_ivp`，并选择隐式变阶变步长 BDF 方法。稀疏 Jacobian 结构、终止事件、分段积分和严格达标整秒验证均属于 BDF 求解流程。

生成的主程序求解脚本如下：

Listing 2: BDF solver core

```

1 def simulate_bdf(
2     n_radial=None, n_axial=None, rtol=BDF_RTOL, max_step=BDF_MAX_STEP,
3     threshold=None, max_duration=None, record_interval=None,
4     stop_at_threshold=True, check_mass=True,
5 ):
6     """第二、三间共用半长度BDF核心；固定时长与达标停止只改变终止方式。"""
7     nr = N_RADIAL if n_radial is None else n_radial
8     nz = N_AXIAL if n_axial is None else n_axial
9     threshold = THRESHOLD if threshold is None else threshold
10    end = MAX_DURATION if max_duration is None else max_duration
11    interval = RECORD_INTERVAL if record_interval is None else record_interval
12    if nr < 2 or nz < 2 or min(rtol, max_step, end, interval) <= 0:
13        raise ValueError("网格至少2，容差、步长和时长必须为正")
14    radii = np.linspace(0, RADIUS, nr+1)
15    axial_positions = np.linspace(0, HALF_LENGTH, nz+1)
16    dr, dz = RADIUS/nr, HALF_LENGTH/nz
17    n = (nr+1)*(nz+1)
18    def residual(t, y):
19        return right_hand_side(y, t, nr, nz, dr, dz)
20    def crossed(t, y):
21        return float(y[n:].max()-threshold)
22    crossed.terminal, crossed.direction = True, -1
23    options = dict(method="BDF", rtol=rtol,
24                    atol=component_atol(n)*(rtol/RTOL), jac_sparsity=
25                    sparsity_pattern(nr,nz),
26                    dense_output=True)
27    boundaries = sorted(set([0., float(end)] + [x for x in
28        ((CRITICAL_TIME, AIR_END_TIME) if not PHASE_BLEND else (AIR_END_TIME,))
29        if 0<x<end]))
30    segments=[]
31    state=make_state(nr,nz)
32    for left,right in zip(boundaries[:-1],boundaries[1:]):
33        def rhs(t,y):
34            tt=np.nextafter(t,-np.inf) if right==CRITICAL_TIME and t>=right else

```

```

t
33         return residual(tt,y)
34         segment=solve_ivp(rhs,(left,right),state,
35                             events=crossed if stop_at_threshold else None,
36                             max_step=min(max_step,ENV_FINE_INTERVAL_S) if left<ENV_FINE_UNTIL_S
else max_step, **options)
37         if not segment.success:
38             raise RuntimeError(segment.message)
39         segments.append(segment)
40         state=segment.y[:,-1]
41         if stop_at_threshold and segment.t_events[0].size:
42             break
43         crossing=None
44         stop=float(end)
45         if stop_at_threshold:
46             if not segments[-1].t_events[0].size:
47                 raise RuntimeError("在允许时长内未达标，不能导出烘干完成结果")
48             crossing=float(segments[-1].t_events[0][0])
49             stop=float(math.floor(crossing)+1)
50             if stop>end:
51                 raise RuntimeError("严格达标整秒超出允许时长")
52             tail=solve_ivp(residual,(crossing,stop),state,max_step=max_step,**
options)
53             if not tail.success or tail.y[n:,-1].max()>=threshold:
54                 raise RuntimeError("严格达标整秒验证失败")
55             segments.append(tail)
56         def state_at(t):
57             for seg in segments:
58                 if t<=seg.t[-1]+1e-9:
59                     return seg.sol(t)
60             raise ValueError("时间超出积分区间")
61         times=np.r_[np.arange(0.,stop,interval),stop]
62         tp,cp=[],[]
63         for t in times:
64             a,b=profiles_from_state(state_at(t),OUTPUT_RADII,radii,axial_positions,
nr,nz)
65             tp.append(a);cp.append(b)
66         final=state_at(stop)
67         if not np.isfinite(final).all() or final[n:].min() < -ATOL_C:
68             raise RuntimeError("终态包含非法数值或负含水率")

```

```

69     balance=mass_balance(segments,n,make_geometry(nr,nz)[2],
70                           lambda t,y:boundary_inflow(t,y,nr,nz)) if check_mass
    else None
71     return dict(time_s=times,radius_cm=OUTPUT_RADII*100,
72                 xi=OUTPUT_RADII/RADIUS,
73                 temperature_c=np.asarray(tp),moisture_kg_per_kg=np.asarray(cp),
74                 final_time_s=stop,final_state=final,radii=radii,axial_positions=
    axial_positions,
75                 n_radial=nr,n_axial=nz,solver="bdf",crossing_time_s=crossing,
76                 reached_threshold=bool(stop_at_threshold),conservation=balance,
77                 steps=1+sum(s.t.size-1 for s in segments),nfev=sum(s.nfev for s
    in segments),
78                 average_iterations=float("nan"))

```

该程序完成了：

1. 构造右端函数和含水量阈值事件；
2. 配置 method="BDF"、相对/绝对容差、稀疏 Jacobian；
3. 在临界时刻和附件数据结束时刻处分段积分；
4. 利用终止事件定位首次达标时刻；
5. 推进到严格达标的整秒并验证；
6. 在 60 s 时间网络上提取结果；
7. 进行全局水分守恒检验。

5.4 论文对应位置

支撑材料 problem III, solve_problem3.py

5.5 人工复核，修改与采纳

1. 检查隐式迭代的收敛容差，最大迭代次数和平均迭代次数
2. 检查在临界时刻和附件数据结束时刻的样例
3. 进行全局水分守恒检验。
4. 检查结果文件的时间单位，径向单位和有效数字是否符合模板
5. 人工修改代码使得代码更加精简，可读性更强
6. 确认结果无误后采纳代码

6 结论

本次数学建模任务以 Codex 作为主要人工智能辅助工具。Codex 在思路整理中帮助判断预热升温和恒温干燥的阶段分界，在图片绘制中生成径向含水量剖面及多组论文图，在代码辅助中实现守恒有限体积离散、自适应 BDF 算法和多项数值检验。需要再次说明的是，Codex 是建模工作流中的辅助工具，最终采用的模型、参数、结果和结论均经过人工判断与数值检验。